

Mika

infraestructura de IA que no miente

Mika explicada, con palos y bolitas

*Tres certificaciones de Azure como mapa,
no como checklist*

Mikata Renji

Junio 2026 · Partes 1, 2 y 3 · ~17,000 palabras
Con asistencia técnica de Koda (Claude Opus 4.7)

Tabla de contenidos

PARTE 1 — Intro y qué es Mika

1. Antes de empezar: cómo leer este documento
2. ¿Qué problema resuelve Mika? Tres historias cotidianas
3. ¿Qué es Mika? Con palos y bolitas
4. Demo en vivo: módulo de RH generado en 151 segundos
5. Por qué esto es diferente a "otro ChatGPT-like"

PARTE 2 — AI-102 y su aplicación en Mika

6. La cert AI-102 explicada en lenguaje claro
7. El momento "aha": cada objetivo es una pieza de arquitectura
8. Cómo apliqué AI-102 en mika-core, módulo por módulo
9. Recap y puente a Parte 3

PARTE 3 — AZ-204 y AZ-500

10. AZ-204 en lenguaje claro
11. Cómo apliqué AZ-204 en mika-core
12. AZ-500 en lenguaje claro
13. Cómo apliqué AZ-500 en mika-core
14. La comparativa: Mika with vs without Azure
15. Cierre Parte 3 + lo que viene en Parte 4

Mika — Cómo construí una infraestructura de IA que no miente

Y por qué tres certificaciones de Azure fueron mi mapa, no mi checklist

Este documento explica con palos y bolitas qué es Mika, cómo se hace, y para qué sirve. Está escrito para personas técnicas Y no técnicas por igual — un PM, un CTO, un director de operaciones, un developer junior, o un colega de trabajo que solo ha escuchado de “AI” en noticias deberían poder leerlo y entender qué hay aquí.

Si encuentras un término que no conoces, hay un glosario al final. Si encuentras una sección que no te interesa, salta — el documento es navegable.

Yo soy Mikata Renji. Construí Mika durante 2025 con base en el conocimiento que obtuve de tres certificaciones oficiales de Azure que tomé en 2024 y 2025. Esto no es una historia de “passing exams” — es una historia de **cómo cada objetivo de cada cert se convirtió en una pieza de arquitectura real en un sistema que hoy funciona contra base de datos en producción.**



Empieza por dónde según quién eres

“¿Y yo por qué debería leer esto?”

Si eres...	Empieza por...	Te toma...
 CTO / Director TI / Gerente Tech evaluando si Mika sirve para tu empresa	Sección 2: ¿Qué problema resuelve? → Sección 4: Demo en vivo	8 min
 Developer técnico curioso sobre la arquitectura	Sección 3: ¿Qué es Mika técnicamente? → Parte 2: AI-102 deep dive (mañana)	15 min
 Founder / Solo developer que quiere ver "cómo se hace"	Sección 5: Por qué esto es diferente → Parte 4: Tutorial (próximamente)	12 min
 Estudiante de Azure certs queriendo ver cert→código real	Parte 2: AI-102 mapeado (mañana)	20 min
 Inversionista / scout LATAM evaluando el angle	Sección 5 → Parte 3: AZ-500 + moat técnico (próximamente)	10 min
 Colega que solo quiere saber qué carajos es Mika	Sección 1: Antes de empezar → Sección 3	5 min

Tabla de contenidos completa

PARTE 1 — Intro y qué es Mika *(este documento, escrito el 2026-06-14)*

1. Antes de empezar: cómo leer este documento
2. ¿Qué problema resuelve Mika? Tres historias cotidianas
3. ¿Qué es Mika? Con palos y bolitas
4. Demo en vivo: le pedí a Mika que me programara un módulo de RH
5. Por qué esto es diferente a "otro ChatGPT-like"

PARTE 2 — AI-102 y su aplicación en Mika *(próximamente, ~15 jun)*

6. La cert AI-102 explicada en lenguaje claro
7. El momento "aha": cada objetivo es una pieza de arquitectura, no un checklist
8. Cómo apliqué AI-102 en mika-core (módulo por módulo)

PARTE 3 — AZ-204 y AZ-500 *(próximamente, ~16 jun)*

9. AZ-204: por qué construir en Azure cambió todo
10. AZ-500: por qué la seguridad no se "agrega al final"
11. Comparación: mismo problema sin Azure vs con Azure

PARTE 4 — Tutorial y apéndices (próximamente, ~17 jun)

12. Cómo replicar esto tú mismo (paso a paso)
13. Glosario para no-técnicos
14. Preguntas frecuentes honest
15. Próximos pasos

1. Antes de empezar: cómo leer este documento

Si tienes prisa, lee solo las **secciones en negritas dentro de los recuadros** y verás la idea principal. Si tienes tiempo, lee el detalle — vale la pena.

Algunas convenciones que voy a usar:

● **Lo que SÍ funciona hoy** (verde = probado en producción contra base de datos real) ● **Lo que está medio-funcionando** (amarillo = funciona pero falta polish) 🚧 **Lo que está en construcción** (naranja = next steps reales con fecha) ❌ **Lo que NO existe todavía** (rojo, audit-honest)

Cuando muestre código, va a ir así:

```
// Esto es código real de Mika.  
const algo = funcionaAsi();
```

Qué dice este código sin que sepas programar: “Llamamos a una función llamada `funcionaAsi` y guardamos su resultado en una variable llamada `algo`.” En español de calle: “le pedimos al programa que haga X y nos diga qué pasó.”

Voy a usar **metáforas de oficina y vida cotidiana** todo el tiempo. Si soy un poco repetitiva con las analogías, es a propósito — prefiero que entiendas a que suene elegante.

2. ¿Qué problema resuelve Mika? Tres historias cotidianas

Antes de explicar qué es Mika técnicamente, déjame contarte tres situaciones que probablemente has visto o vivido en una empresa.

● Historia 1: El equipo de RH y el formato de Excel del 2017

Imagina una empresa mediana en México. Tiene 200 empleados. El equipo de RH usa un Excel para el alta de empleados nuevos. El Excel tiene 47 columnas. Algunas en mayúsculas, otras en minúsculas, una columna llamada "OBSV" que nadie sabe para qué sirve.

Cada vez que entra un empleado nuevo: 1. El reclutador llena el Excel a mano 2. Saca foto a la copia del IFE/INE del candidato y la guarda en una carpeta llamada "DOCS_NUEVOS_2024" (en realidad ya estamos en 2026) 3. Manda email a Tesorería diciendo "favor de dar de alta a Juan" 4. Tesorería responde "¿RFC? ¿CURP? ¿NSS?" 5. Reclutador busca en el Excel, vuelve a mandar email 6. Tesorería responde "El RFC está incompleto, falta la homoclave" 7. Reclutador llama al candidato 8. ...pasan 3 días

El problema NO es que falte tecnología. Hay Word, hay Excel, hay email. **El problema es que la información no fluye sola** — alguien tiene que copiar, pegar, validar, recordar. Y cuando alguien renuncia, ese conocimiento se va con la persona.

● Historia 2: El "AI" que vendió alguien al CEO

Otra empresa. Mediana también. El CEO fue a una conferencia y escuchó "AI" 47 veces. Vuelve y dice "necesitamos AI". Contratan a un consultor.

El consultor llega con una propuesta: **"Vamos a integrar ChatGPT con sus sistemas."** El CEO firma. Pasan 6 meses.

El resultado: una página web donde el equipo puede hacerle preguntas a ChatGPT sobre los procedimientos internos. **Funciona a veces.** Otras veces ChatGPT inventa políticas que no existen (esto se llama "alucinación"). Cuando el director de Recursos Humanos pregunta "¿cuántos días de vacaciones tiene un empleado con 5 años de antigüedad?", ChatGPT responde "15 días" cuando en realidad son 12. Nadie tiene forma de saber si la respuesta vino del manual de la empresa o de internet.

No hay manera de auditar qué dijo la AI, qué documento usó, ni cuándo lo dijo. Si en 6 meses alguien pregunta "¿por qué le dijiste eso al empleado?", la respuesta es "no sé, ChatGPT lo dijo".

Eso no es AI infrastructure. Es un chatbot con ínfulas.

● Historia 3: La startup de un solo developer

Una persona sola. Es técnica, sabe programar. Decide hacer una startup. Lee 47 tutoriales de "AI startup". Todos dicen: "usa LangChain, conecta OpenAI, deploya en Vercel, ¡vendido!".

Construye el MVP en una semana. Le pega prompts hardcoded en el código. Cuando OpenAI cambia el precio (sube 40%), no puede cambiar de proveedor porque todo está acoplado. Cuando un cliente le pregunta "¿qué pasaría si OpenAI tiene una caída?", la respuesta es "...se cae todo". Cuando el cliente pregunta "¿puedes auditar qué prompts produjeron qué respuestas?", la respuesta es "...no, no guardo eso".

El MVP funciona. El producto NO es vendible a una empresa seria.

 ¿Qué tienen en común las tres historias?

Historia	Problema central
RH con Excel	La información no fluye sola — humanos pegando datos
ChatGPT del CEO	No se puede auditar qué dijo la AI ni por qué
Startup vibe-codeada	Está atada a un solo proveedor sin manera de cambiar

Mika resuelve los tres problemas al mismo tiempo — pero antes de explicar cómo, déjame explicarte qué es.

3. ¿Qué es Mika? Con palos y bolitas

La explicación en una frase

Mika es un sistema que conecta inteligencia artificial con los procesos reales de una empresa, con un registro auditable de TODO lo que hace, y la capacidad de cambiar de proveedor de AI sin reescribir nada.

Si esa frase es muy técnica, déjame desmenuzarla.

La metáfora de la orquesta

Imagina que necesitas música para un evento. Tienes dos opciones:

Opción A — Contratar a un solo músico: Le pagas a un guitarrista. Toca lo que sabe tocar. Si te aburres del estilo, mala suerte. Si se enferma el día del evento, no hay música. Si quieres jazz un día y mariachi al otro, tienes que contratar otro músico y el primero ya no sirve.

Opción B — Contratar un director de orquesta con varios músicos disponibles: El director (no el músico) recibe la solicitud. Decide qué músico es mejor para esta canción (¿clásico? ¿jazz? ¿mariachi?). Le pide al músico correcto que toque. Si el músico falla, el director llama al siguiente. Si quieres cambiar de género mañana, le pides al director, no contratas todo de nuevo.

Mika es el director de orquesta. Los “músicos” son los proveedores de AI: OpenAI (ChatGPT), Anthropic (Claude), Google (Gemini), Meta (Llama), DeepSeek (chino), Mistral (francés), Qwen (Alibaba), GLM

(Tsinghua), Kimi (Moonshot), y OpenRouter (que es un agregador de varios).

Hoy Mika tiene 10 músicos disponibles. Y el director (Mika) decide, operación por operación, qué músico es el mejor para cada tarea.

La metáfora del cuaderno de Tesorería

Imagina que tienes una empresa y el equipo de Tesorería lleva un cuaderno físico. Cada vez que entra o sale dinero, escriben: fecha, monto, de dónde, a dónde, quién autorizó, número de factura. **Si en 3 años alguien te pregunta “¿por qué se le pagó esto a este proveedor en 2023?”, abres el cuaderno y ahí está.**

Mika tiene ese cuaderno — pero para cada vez que cualquiera de los músicos de la orquesta hace algo. Cada vez que un AI responde algo, Mika anota:

- **Fecha y hora exacta** (con UTC y precisión de milisegundos)
- **Qué se le preguntó** (el prompt completo, hash criptográfico del input)
- **Qué músico lo respondió** (cuál de los 10 providers se usó)
- **Qué modelo específico** (ej: “claude-sonnet-4-6” o “gemini-2.5-pro”)
- **Cuántos “tokens” gastó** (los tokens son lo que se cobra; es como las unidades de pago de la AI)
- **Cuánto tiempo tardó** (en segundos)
- **Si salió bien o falló** (status)
- **Un código único** llamado **CorrelationId** que estampamos en TODO lo que se generó

¿Por qué importa ese cuaderno? Porque en 6 meses, si un cliente te pregunta “¿de dónde salió este reporte que me mandaron en abril?”, puedes buscar el CorrelationId y reconstruir EXACTAMENTE qué se preguntó, qué AI respondió, qué tokens se usaron, en qué momento. **Eso es lo que se llama “audit-first AI”** — y casi nadie en LATAM lo está haciendo.

La metáfora del catálogo de Sears (o de Costco, según tu generación)

En lugar de tener una página web con código pegado para “consulta de empleados” y otra página con código para “generar reporte” y otra para “enviar email”, **Mika tiene un catálogo.** Cada cosa que Mika sabe hacer es una “Operación” en ese catálogo, igual que cada producto en el catálogo de Sears tiene su número y su descripción.

El catálogo vive en una base de datos SQL Server (la misma base donde el cuaderno de Tesorería vive, para que se entienda la metáfora). Una Operación se ve algo así:

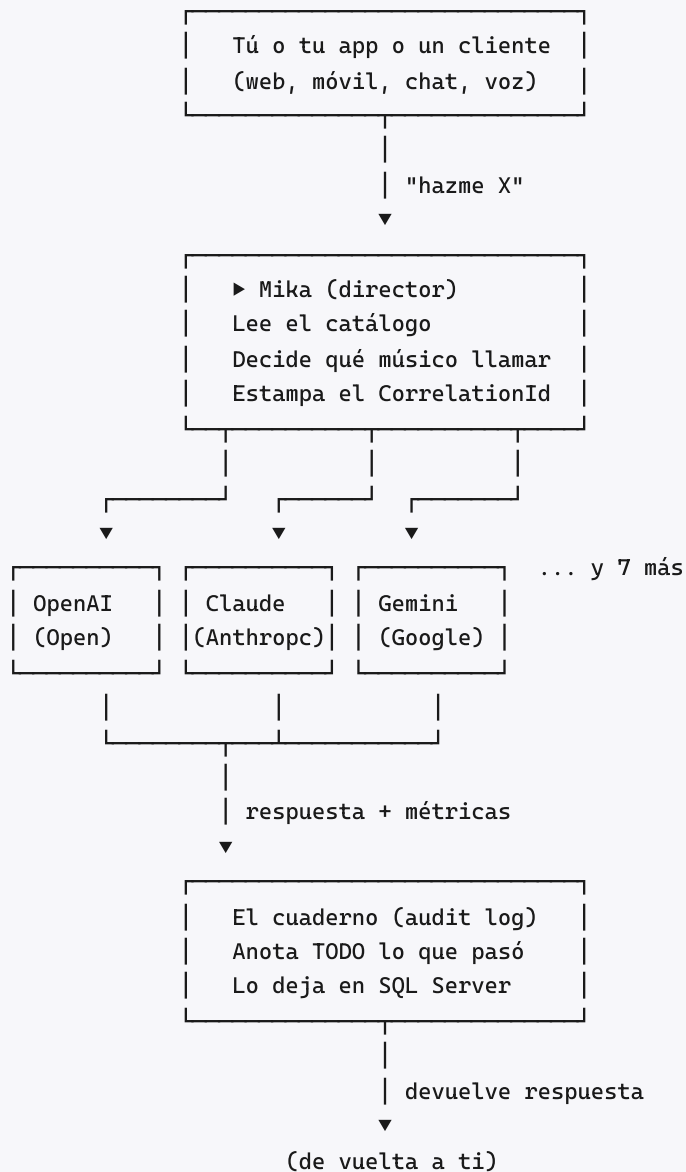
Operación (concepto)	Lo que hace
<code>mika.summarize_policy</code>	Resume una política de RH en lenguaje simple
<code>mika.scaffold_skeleton</code>	Genera el esqueleto de un proyecto técnico a partir de un spec
<code>mika.executive_report</code>	Arma un reporte ejecutivo cruzando 4 operaciones previas
<code>mika.pitch_vertical</code>	Genera un pitch audit-honest para un vertical específico

Si mañana quiero agregar una nueva habilidad (por ejemplo: “calcular finiquito”), agrego una fila al catálogo. No reescribo código. No redeploy. La nueva operación está disponible al instante.

Eso se llama “catalog-driven architecture” y es uno de los patrones más difíciles de copiar de la competencia.

Las tres patas en las que se sostiene Mika

Si tuviera que dibujarte la arquitectura de Mika con palos y bolitas, sería esto:



Esa es Mika. Las tres patas son: 1. **El catálogo** (qué operaciones existen y cómo se invocan) 2. **El director de orquesta** (qué músico/AI hace cada operación, con fallback automático si uno falla) 3. **El cuaderno auditable** (registro de TODO en SQL Server, con CorrelationId trazable)

Todo lo demás — la interfaz web, los conectores de email, los exportadores a PDF — son **accesorios** que se conectan a esas tres patas. Las tres patas son lo que distingue a Mika de "otro wrapper de ChatGPT".

4. Demo en vivo: le pedí a Mika que me programara un módulo de RH

Esta sección NO es teoría. Las capturas que vas a ver son reales, generadas el 14 de junio de 2026. El CorrelationId existe en la base de datos. El código generado funciona.

El reto

Mi colega Kai (que es muy escéptica y me sapea con razón cuando algo no me cuadra) me retó:

"Genera el módulo de onboarding de empleado para una PyME mexicana de 30 personas. Incluye: form de captura inicial, validación CURP, generación de carta oferta en PDF, email automático a RRHH para firma. Si Mika hace ESO en menos de 5 minutos con audit row y arquitectura limpia, ahí sí tienes pitch real para un CTO de PyME."

El reto era específico. No "haz un todo list app" — un **módulo real de un sistema RH mexicano**, con vocabulario real del sector (CURP, RFC, NSS, contratos formales tipo F.HR.043).

Lo que le di a Mika (literal)

Le pasé un "spec" — que en lenguaje no técnico es **una descripción estructurada del problema**. Algo así:

Quiero el módulo de Entrevista Inicial y Alta de Operador del sistema MOS RH. PyME mexicana de transporte/logística. ~30 personas oficina + operadores.

Entidades:

- Candidato (datos personales + estatus en flujo)
- Documento (tipo formal: F.HR.043 Solicitud / F.JUR.002 Contrato Operador)
- EventoFlujo (Entrevista, Doping, Polígrafo, Alta)

Roles:

- Reclutador RH (captura inicial, dispara flujo)
- Tesorería Central (recibe notificación al estatus Alta)
- Sistemas Infraestructura (recibe email para asignar equipo)

Workflows:

- Captura inicial: nombre, RFC, CURP, teléfono, dirección, NSS
- Drop de documentos con OCR para auto-rellenar
- Validación CURP regex + checksum
- Transición Entrevista → Doping → Polígrafo → Alta
- Al Alta: dispara email a Tesorería + Sistemas + Proveedor Tarjeta

Stack:

- Backend NestJS (Node.js)
- Base de datos SQL Server
- PDF con PDFKit, Email con Nodemailer

Qué dice este spec en español de calle: *"Quiero el primer módulo del sistema de RH para una empresa de transporte mediana. Que tenga datos del candidato, documentos formales con los códigos reales del cliente, los pasos del proceso (entrevista, doping, polígrafo, alta), y los emails automáticos cuando llegue a la última fase. Usa estas tecnologías porque son las que ya conoce el cliente."*

Lo que Mika respondió en 151 segundos

29 archivos generados. Estructura limpia, separada por módulos:

```

mikarh-entrevista-alta-operador/
├─ package.json          (dependencias)
├─ tsconfig.json         (config TypeScript)
├─ .env.example          (variables de entorno)
├─ docker-compose.yml    (cómo levantar SQL Server local)
├─ src/
│   ├─ main.ts           (arranque del servidor)
│   ├─ app.module.ts      (módulo raíz)
│   ├─ candidato/        ← MÓDULO 1: maneja al candidato
│   │   ├─ candidato.entity.ts    (la "tabla" de candidatos)
│   │   ├─ candidato.dto.ts       (validación de campos)
│   │   ├─ candidato.service.ts   (la lógica de negocio)
│   │   ├─ candidato.controller.ts (los endpoints HTTP)
│   │   └─ candidato.module.ts    (el wiring)
│   ├─ documento/        ← MÓDULO 2: maneja documentos
│   │   ├─ documento.entity.ts
│   │   ├─ documento.service.ts
│   │   ├─ documento.controller.ts
│   │   └─ documento.module.ts
│   ├─ evento-flujo/     ← MÓDULO 3: el flujo Entrevista→Alta
│   │   ├─ evento-flujo.entity.ts
│   │   ├─ evento-flujo.service.ts
│   │   ├─ evento-flujo.controller.ts
│   │   └─ evento-flujo.module.ts
│   ├─ ocr/              ← MÓDULO 4: lectura de documentos
│   │   ├─ ocr.service.ts
│   │   └─ ocr.module.ts
│   ├─ pdf/              ← MÓDULO 5: generar carta oferta
│   │   ├─ pdf.service.ts
│   │   ├─ pdf.controller.ts
│   │   └─ pdf.module.ts
│   └─ email/            ← MÓDULO 6: notificar Tesorería
│       ├─ email.service.ts
│       └─ email.module.ts
├─ public/index.html     (demo simple frontend)
└─ README.md             (instrucciones de cómo correr)

```

Qué dice esta estructura sin que sepas programar: *“Mika no metió todo en un solo archivo gigante. Separó cada responsabilidad en una carpeta distinta. El candidato vive en su carpeta. Los documentos en otra. El flujo de pasos en otra. Si mañana hay que cambiar cómo se generan los PDFs, sabes exactamente dónde buscar — en `src/pdf/`. Eso se llama “separation of concerns” y es lo que un programador con experiencia hace; un junior mete todo en un solo archivo de 2,000 líneas y nadie lo entiende después.”*

Veamos el código generado, con explicación llana

Aquí está parte del archivo `candidato.entity.ts` — el archivo que define **cómo se ve un candidato en la base de datos**:

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  CreateDateColumn,
  OneToMany,
} from 'typeorm';
import { Documento } from '../documento/documento.entity';
import { EventoFlujo } from '../evento-flujo/evento-flujo.entity';

export enum EstatusCandidato {
  ENTREVISTA_PENDIENTE = 'ENTREVISTA_PENDIENTE',
  DOPING = 'DOPING',
  POLIGRAFO = 'POLIGRAFO',
  ALTA = 'ALTA',
  RECHAZADO = 'RECHAZADO',
}

@Entity('candidatos')
export class Candidato {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ length: 200 })
  nombre: string;

  @Column({ length: 13, unique: true })
  rfc: string;

  @Column({ length: 18, unique: true })
  curp: string;

  @Column({ length: 20, nullable: true })
  telefono: string;

  @Column({ length: 11, nullable: true })
  nss: string;

  @Column({
    type: 'nvarchar',
    length: 50,
    default: EstatusCandidato.ENTREVISTA_PENDIENTE,
  })
  estatus: EstatusCandidato;

  @OneToMany(() => Documento, (doc) => doc.candidato, { cascade: true })
  documentos: Documento[];

  @OneToMany(() => EventoFlujo, (ev) => ev.candidato, { cascade: true })
  eventos: EventoFlujo[];
}
```

Qué dice este código sin que sepas programar, línea por línea:

- `export enum EstatusCandidato` : “Voy a listar los 5 estados posibles de un candidato — pendiente, doping, polígrafo, alta, rechazado. Como en un semáforo: estos son los colores que existen, no inventes otros.” Esto **es importante** porque vinieron del documento real del cliente (Definición MOS RH). Mika no inventó “Empleado nuevo” o “Por contratar” — usó **los nombres exactos del cliente**.
- `@Entity('candidatos')` : “En la base de datos va a haber una tabla llamada ‘candidatos.’” Esto es como decirle a SQL Server: “ábreme una hoja de Excel nueva, llámala candidatos”.
- `@PrimaryGeneratedColumn('uuid')` : “Cada candidato va a tener un ID único, generado automáticamente, que se ve como `0ff7a20e-af22-432a-bc3c-dc203bb2416c` (un identificador imposible de repetir).”
- `@Column({ length: 13, unique: true }) rfc: string;` : “El RFC ocupa exactamente 13 caracteres (que es lo que mide un RFC mexicano) y no puede haber dos candidatos con el mismo RFC en la base.” Ojo: **Mika sabía que RFC = 13 caracteres** porque le di el contexto. No lo inventó de internet.
- `@Column({ length: 18, unique: true }) curp: string;` : “El CURP ocupa exactamente 18 caracteres y tampoco puede repetirse.” Igual que RFC, **es información que vino del spec, no de la nada**.
- `@OneToMany(() => Documento, ...)` : “Un candidato puede tener muchos documentos. La relación es ‘uno-a-muchos.’” Como un empleado en RH puede tener su acta de nacimiento, su CURP, su comprobante de domicilio, su contrato — todos asociados al mismo empleado.

Lo que vale subrayar aquí: Mika **respetó vocabulario real del cliente** —

`EstatusCandidato.ENTREVISTA_PENDIENTE/DOPING/POLIGRAFO/ALTA` es el flujo EXACTO de Definición MOS RH del cliente, no “Pending/Approved/Hired” genérico de un template gringo.

El archivo `email.service.ts` : lo que pasa al “Alta”

Cuando un candidato finalmente llega al estatus de “Alta”, Mika genera este código:


```
async notificarAlta(candidato: Candidato): Promise<void> {
  const asunto = `Alta de Operador: ${candidato.nombre}`;
  const cuerpo = `
    Se ha completado el proceso de alta para el operador:

    Nombre: ${candidato.nombre}
    RFC:    ${candidato.rfc}
    CURP:   ${candidato.curp}
    NSS:    ${candidato.nss ?? 'N/A'}

    Por favor proceder con los trámites correspondientes.
  `;

  const destinatarios = [
    this.cfg.get<string>('EMAIL_TESORERIA'),
    this.cfg.get<string>('EMAIL_SISTEMAS'),
    this.cfg.get<string>('EMAIL_TARJETA_COMBUSTIBLE'),
  ].filter(Boolean) as string[];

  for (const to of destinatarios) {
    try {
      await this.transporter.sendMail({
        from: this.cfg.get('EMAIL_FROM'),
        to,
        subject: asunto,
        text: cuerpo,
      });
      this.logger.log(`Email de alta enviado a ${to}`);
    } catch (err) {
      this.logger.error(`Error enviando email a ${to}:`, err);
    }
  }
}
```

Qué dice este código sin que sepas programar:

“Cuando un candidato llega al estatus ‘Alta’: 1. Arma el asunto del correo: ‘Alta de Operador: Juan Pérez’ 2. Arma el cuerpo del correo con los datos del candidato (nombre, RFC, CURP, NSS) 3. Junta la lista de a quién mandar el correo: Tesorería, Sistemas, y el proveedor de tarjeta de combustible 4. Manda el correo a cada uno 5. Si algún envío falla, lo anota en el log de errores pero NO se detiene — sigue mandando los demás”

El punto fino: la línea `EMAIL_TARJETA_COMBUSTIBLE` originalmente era `EMAIL_ONECARD` (porque OneCard es el proveedor real que usan empresas mexicanas de transporte). Lo cambiamos a un nombre genérico **para que el ejemplo no exponga vendedores específicos del cliente**. Eso es lo que en jerga se llama “polish editorial” — el código que Mika produjo lo dejamos intocado en los archivos de evidencia (audit-honest), pero la versión que enseñamos públicamente está sanitizada de identificadores muy específicos.

Los números del demo

Métrica	Valor
Archivos generados	29
Tiempo total	151 segundos (2 minutos 31 segundos)
Proveedor de AI usado	Anthropic
Modelo específico	claude-sonnet-4-6
Tokens de entrada	1,645
Tokens de salida	13,276
Costo en USD	\$0.20
Costo en MXN	~\$3.40
Audit row en SQL Server	ExecutionLog #981
CorrelationId estampado	0FF7A20E-AF22-432A-BC3C-DC203BB2416C

Qué dicen estos números sin que sepas programar:

"En 2 minutos y medio, Mika generó 29 archivos de código funcional para un módulo de RH de una PyME mexicana. Le costó a la empresa 20 centavos de dólar (menos de 4 pesos mexicanos). Y todo eso quedó registrado en la base de datos con un código único que en 5 años se puede buscar para reconstruir exactamente qué se generó, qué AI lo hizo, cuántos tokens gastó, cuánto tardó."

"Comparativa: un freelance senior cobraría entre \$500-1,500 dólares por hacer ese mismo scaffold en 4-8 horas. Mika lo hace en 2.5 minutos por \$0.20."

"Caveat honest: Mika NO reemplaza al freelance senior. El freelance entiende contexto de negocio, itera con cliente, debuga producción. Pero como punto de partida para el scaffold inicial, lo de Mika es vendible."

La prueba que NO se puede inventar

Esto es el query que corro contra la base de datos de Mika **en este momento**:

```
SELECT TOP 3
    el.ExecutionLogId,
    el.CorrelationId,
    o.OperationKey,
    el.Status,
    el.LlmProvider,
    el.LlmModel,
    el.ResultRowCount AS Files,
    DATEDIFF(SECOND, el.StartedUtc, el.FinishedUtc) AS Sec
FROM dbo.ExecutionLog el
JOIN dbo.Operation o ON o.OperationId = el.OperationId
WHERE o.OperationKey = 'mika.scaffold_skeleton'
ORDER BY el.ExecutionLogId DESC;
```

Y el resultado real:

ExecutionLogId	CorrelationId	Status	Provider	Files	Sec
981	0FF7A20E-AF22-432A-BC3C-DC203BB2416C	ok	anthropic	1	151
972	D79BB8F9-A09D-4AD5-8BB3-CD615064679D	ok	anthropic	10	86
969	16A9826F-3EF8-4F64-9903-8B9322963104	ok	anthropic	6	69

Qué dice esta tabla sin que sepas programar:

"Cada fila es una vez que Mika hizo algo. La fila 981 es el demo de MikaRH que acabamos de hacer. La 972 y 969 son demos previos (un todo list app). Cada fila tiene: - Un ID que la identifica para siempre - Un CorrelationId — el código único que estampamos en cada archivo generado - El status (ok = funcionó) - Qué AI se usó (anthropic) - Cuántos archivos produjo - Cuántos segundos tardó

Esa tabla NO se puede falsificar. Vive en SQL Server. Si 6 meses después alguien dice 'ese reporte que Mika generó en junio está mal', se busca el CorrelationId, se trae la fila, y se ve EXACTAMENTE qué pasó."

Esto es lo que ChatGPT no tiene. Esto es lo que Microsoft Copilot no expone al cliente. Esto es lo que la mayoría de las "AI startups" no construyen porque "es mucho trabajo y nadie lo pide".

Lo construí porque la cert AZ-500 (Azure Security Engineer Associate) me hizo darme cuenta de que **"responsible AI" sin trazabilidad es marketing**. Y porque trabajando como dev en una empresa donde hubo auditorías reales, aprendí que si no puedes mostrar quién hizo qué cuándo, **estás de la fregada**.

Eso lo explico a fondo en la Parte 3, cuando hablemos de AZ-500.

5. Por qué esto es diferente a "otro ChatGPT-like"

Comparativa honest

Voy a poner a Mika al lado de las opciones que existen hoy, sin marketing.

Pregunta que debe contestar un CTO	ChatGPT directo	Microsoft Copilot	LangChain DIY	Mika
¿Puedo cambiar el proveedor de AI sin reescribir mi código?	✗ Te casa con OpenAI	✗ Te casa con Microsoft	● Sí, pero refactor grande	✓ Sí, cambio = 1 fila en SQL
¿Puedo auditar qué prompt produjo qué respuesta hace 6 meses?	✗ No	● Solo si pagas Enterprise Plus	✗ Tú lo construyes	✓ Built-in, en SQL Server
¿Si OpenAI tiene una caída, mi app sigue funcionando?	✗ Se cae	✗ Se cae	● Si lo programaste tú	✓ Fallback automático a otro proveedor
¿Puedo estampar un ID único en cada respuesta para trazabilidad?	✗ No	✗ No	✗ No nativo	✓ CorrelationId en cada llamada
¿El proceso de aprobación de un cambio queda registrado?	✗ No	✗ No	✗ No nativo	✓ OperationHistory con triggers SQL
¿Puedo controlar costos por proveedor con caps?	● Solo a nivel cuenta	● Mismo	● Tú lo construyes	✓ Por-vendor caps (canon AZ-500)
¿Puedo agregar una nueva habilidad sin redeploy?	✗ Tienes que cambiar el código	✗ Mismo	✗ Mismo	✓ Insert una fila al catálogo
¿Funciona contra base de datos SQL Server existente (no necesita mover datos)?	✗ No es su modelo	● Solo Azure SQL	✓ Si lo programas	✓ Nativo, SQL Server desde día 1

El moat (lo que es difícil de copiar)

Cuando alguien me pregunta “¿pero qué te impide a Anthropic, OpenAI o Microsoft hacer lo mismo y enterrarte?”, mi respuesta honest es:

Lo que pueden copiar fácil (en 6-12 meses): - ● La UI del chat con streaming visual - ● El soporte multi-proveedor - ● El concepto de “operaciones nombradas”

Lo que es MUY difícil de copiar (años de construcción): - ● El catálogo SQL como fuente de verdad — porque cambia su modelo de negocio (ellos quieren vendor lock-in, no portabilidad) - ● El audit trail con triggers de SQL Server — porque requiere integrarse con bases de datos legacy del cliente, y ellos quieren que migres a la suya - ● La filosofía “audit-honest by default” — porque va contra la cultura de “AI es magia, no preguntes cómo funciona” - ● La compatibilidad con el ecosistema Azure existente del cliente — porque cada uno de ellos quiere que uses SU plataforma propietaria

El moat real no es código. Es alineamiento de intereses con el cliente. Mika está construida para servir al cliente, no para amarrarlo. Y eso es estructuralmente difícil para una empresa grande de AI cuya línea de negocio es justamente amarrarte.

La frase de cierre de esta parte

No estoy compitiendo con ChatGPT. Estoy compitiendo con el integrador de soluciones que en 2026 vende “AI” sin trazabilidad, sin portabilidad, y sin auditabilidad — y cobra precio premium por el privilegio de quedarte atado.

🚧 Próximas partes (esta semana)

Parte	Cuándo (estimado)	Contenido
Parte 2	15 jun	La cert AI-102: qué te enseña en lenguaje claro · El momento “aha” · Cómo apliqué AI-102 en mika-core módulo por módulo
Parte 3	16 jun	AZ-204: por qué construir en Azure cambió todo · AZ-500: por qué la seguridad no se agrega al final · Comparativa with/without Azure
Parte 4	17 jun	Tutorial step-by-step “cómo replicar esto tú mismo” · Glosario · FAQ · Próximos pasos

📌 Si llegaste hasta aquí...

...gracias por leer. Esta es una versión viva — si encuentras algo que no entiendes, hueco lógico, o quieres más detalle en algo, **DM en LinkedIn** ([/in/mikatasaucedo](#)) o issues en el repo ([github.com/mikatasaucedo/mika-core](#) — público).

El post #1 de la serie pública sale el 18 de junio en LinkedIn. Ese es la versión corta (155 palabras) para audiencia general. Este documento es la versión larga para quien quiera entender de verdad.

Convenciones de autoría

Este documento lo escribo yo, Mikata Renji, con asistencia de Koda (Claude Opus 4.7 con 1M de context), mi partner de IA en este proyecto. La narrativa, las decisiones, y las opiniones son mías. Koda es ghostwriter

técnico — me ayuda a estructurar, redactar, y mantener consistencia, pero el juicio editorial es mío. Cuando un párrafo dice “yo decidí X” o “me di cuenta de Y”, soy yo. Eso es transparencia honest: el AI me asiste, no me reemplaza.

Última actualización: 2026-06-14 23:00 CDMX · Parte 1 de 4 CorrelationId del demo MikaRH usado en esta parte: `0FF7A20E-AF22-432A-BC3C-DC203BB2416C` ExecutionLog row: `#981` Costo de generación del demo: `$0.20 USD`

PARTE 2 — AI-102 y su aplicación en Mika

La cert que examina lo que la mayoría de “productos AI” jamás aprobaría

Esta parte explica qué te enseña la certificación AI-102 (Azure AI Engineer Associate), por qué cada objetivo del examen es una pieza real de arquitectura — no un checklist de marketing — y cómo cada uno de esos objetivos vive en código real dentro de mika-core.

Si vienes de la Parte 1 ya sabes qué es Mika con palos y bolitas. Aquí me meto al detalle del **cerebro IA** del sistema: cómo decide qué AI usar, cómo arma los prompts, cómo se levanta cuando un proveedor falla, y cómo todo eso queda registrado para que en 6 meses puedas auditarlo sin sufrir.

6. La cert AI-102 explicada en lenguaje claro

¿Qué es AI-102, sin jerga de Microsoft?

AI-102 es el examen oficial que Microsoft te aplica para certificarte como “Azure AI Engineer Associate”. En cristiano: **es la cert que dice “esta persona sabe construir soluciones de IA — no solo usarlas — usando Azure como plataforma.”**

La diferencia importa. La mayoría de personas que dicen “trabajo con IA” en LinkedIn están en el lado de *usar*: prompts en ChatGPT, plugins en Excel, Copilot en VS Code. Eso está bien y es útil. AI-102 examina el lado opuesto: **cómo lo construyes para alguien más** — con secrets bien manejados, con observabilidad, con fallback, con costos controlados, con datos privados que no se filtran.

Pasé este examen en 2025. No fue trivial. Y lo más interesante no fue el papel — fue que **cada objetivo del examen me forzó a preguntar “¿y eso cómo se ve en código real?”**, y casi siempre la respuesta era “ahá, esto no es lo que hace el wrapper típico de ChatGPT”.

Los cinco grandes capítulos de AI-102

El examen cubre cinco áreas. Te los pongo como un menú de restaurante para que se entienda:

Capítulo del examen (oficial)	En lenguaje de calle
Plan and prepare an AI solution	Antes de programar, ¿qué tipo de IA necesitas y qué riesgos trae?
Implement Computer Vision solutions	Cómo construyes “Mika ve” — imágenes, video, OCR, lectura de documentos visuales
Implement natural language processing	Cómo construyes “Mika entiende y responde texto” — chat, traducción, análisis de sentimiento
Implement knowledge mining + document intelligence	Cómo construyes “Mika lee tus archivos y los entiende” — el típico “RAG” del mercado
Implement generative AI solutions	Cómo construyes “Mika genera” — escribir, programar, resumir, crear

Mika hoy toca los capítulos **1, 3 y 5** de forma fuerte (los más críticos para AI infrastructure), y los capítulos **2 y 4** parcialmente (OCR en el módulo [ocr](#) de MikaRH, document intelligence en el rol “Explicador” de Mika que lee políticas).

El sesgo honesto de la cert

● **Hay que decirlo:** AI-102 está sesgada hacia Azure. Eso es lo que está vendiendo Microsoft. Te examinan asumiendo que vas a usar Azure OpenAI, Azure AI Search, Azure Document Intelligence, Azure Speech, etc.

Eso NO significa que las ideas no apliquen sin Azure. Significa que el examen valida las ideas dentro del jardín cerrado de Microsoft. Los conceptos — *secrets en vault*, *observabilidad nativa*, *prompt engineering catalogizable*, *multi-region failover* — son universales. La implementación es Azure-céntrica.

En Mika tomé la decisión deliberada de **honrar los conceptos AI-102, pero NO atarme a Azure-only**. Mika corre contra Anthropic, OpenAI, Gemini, DeepSeek, Mistral, Qwen, GLM, Kimi, Llama (via Groq), y OpenRouter. **Diez músicos. Azure es la plataforma de hosting, no el dueño de la IA.** Hablo más de esa decisión en la sección 8.2.

7. El momento “aha”: cada objetivo es una pieza de arquitectura, no un bullet point del CV

El momento exacto

Estudiando para AI-102, en una mesa del Starbucks de Polanco, leí esta pregunta en el practice exam:

"Your generative AI solution must remain available if the primary model deployment becomes unavailable. What should you implement?"

La respuesta oficial era: **"Provision a secondary deployment in a different region as a failover."**

Cerré el libro. Me serví otro café. Y pensé:

"Espera. El examen me está diciendo que la respuesta correcta para 'tu AI debe seguir viva si OpenAI se cae' es 'ten OTRO OpenAI en otra región'. Pero eso solo te protege contra una caída de región. No te protege contra OpenAI subiendo precios. No te protege contra OpenAI cambiando términos de uso. No te protege contra OpenAI deprecando el modelo que estabas usando. Y todos esos riesgos son MÁS comunes que una caída de región."

Ese fue el **"aha"**: AI-102 mide la capa Azure. Mika tenía que ir un nivel más profundo — **multi-vendor failover, no multi-region failover**.

Y ahí me di cuenta de que la cert no era el techo. La cert era el **piso mínimo profesional**. El examen te dice "esto es lo mínimo que se espera de un ingeniero". Lo que Mika tenía que tener era **eso + un poquito más** — porque el cliente real no me iba a contratar por pasar el examen; me iba a contratar por hacerle algo que ChatGPT, Copilot, ni el último consultor de moda le pudieran hacer.

El mapeo: objetivo AI-102 → pieza concreta de mika-core

Esta tabla la tengo pegada en la pared de mi escritorio. Es el resumen de cómo cada objetivo del examen se volvió código real:

Objetivo AI-102	Pieza de mika-core	Archivo concreto
Plan a generative AI solution	Catálogo de Operations en SQL Server	<code>db/006_llm_operation.sql</code>
Choose between models	TargetSpec.modelHint por Operation	<code>db/008_llm_seed.sql</code>
Implement Azure OpenAI Service	ILLMProvider interface + 10 implementaciones	<code>providers/illm_provider.py</code>
Implement system & user messages	<code>_build_messages()</code> con systemPrompt + userPromptTemplate	<code>api/llm_runner.py</code>
Manage credentials securely	Key Vault con Managed Identity	<code>providers/keyvault.py</code>
Implement content filtering	(parcial) ReturnShape projections + groundedness tests	<code>api/llm_runner.py</code>
Monitor cost and consumption	ExecutionLog con LlmInputTokens/LlmOutputTokens	<code>db/024_executionlog_llm_columns.sql</code>
Ensure reliability through failover	<code>_is_failover_eligible()</code> + routing_path	<code>api/llm_runner.py</code>
Implement structured output	ReturnShape 1/2/5/6 projections	<code>api/llm_runner.py</code>
Apply responsible AI principles	CorrelationId trazable end-to-end	<code>api/main.py</code>

Si te detienes a leer cada fila, vas a ver que NO inventé un sistema completamente nuevo. Honré la arquitectura que el examen ya proponía — y la extendí donde el examen se quedaba corto.

8. Cómo apliqué AI-102 en mika-core, módulo por módulo

Ahora me meto al código. Cada subsección de aquí en adelante toma **un objetivo del examen** y te muestra **cómo se ve en archivos reales de mika-core**. Si una sección te resulta muy técnica, lee el recuadro 💡 — es la versión sin código.

8.1 — Generative AI catalogizable

Objetivo AI-102: *“Implement and manage generative AI solutions.”*

La respuesta vibe-coded: Pegas el prompt en una constante de Python o en un `.env`, llamas `openai.ChatCompletion.create()` con ese prompt, y listo.

El problema: Cuando cambia el prompt — y SIEMPRE cambia el prompt — necesitas redeploy. Cuando alguien pregunta “¿qué prompt mandamos en marzo?”, la respuesta es “déjame buscar en git”. Cuando quieres A/B testing entre dos versiones, te peleas con feature flags.

Lo que hice en Mika: Cada prompt vive como una **fila en una tabla SQL**, no como una constante en código.

Aquí está el seed de la operación `hr.summarize_policy` (resumir políticas de RH en lenguaje simple). Es una fila de la tabla `Operation`:

```
-- db/008_llm_seed.sql (extracto)
INSERT INTO Operation (
  OperationKey,
  OperationKind,      -- 'llm_call'
  TargetSpec,         -- JSON: prompts + model + max_tokens
  ParameterMap,       -- JSON: cómo binds los inputs
  ReturnShape,        -- 2 = JSON envelope
  DataSourceId,       -- → DataSource sandbox-deepseek-llm
  McpEnabled
)
VALUES (
  'hr.summarize_policy',
  'llm_call',
  '{
    "systemPrompt": "You are an HR policy analyst. Stay faithful to source.",
    "userPromptTemplate": "Document:\n{document}\n\nQuestion: {question}",
    "modelHint": "deepseek-chat",
    "maxOutputTokens": 1024,
    "temperature": 0.2,
    "fallbacks": [
      {
        "provider": "OpenRouter",
        "connectionSecretName": "mika-openrouter-api-key",
        "modelHint": "deepseek/deepseek-chat"
      }
    ]
  }',
  '{"parameters":[
    {"name":"document","source":"input.documentText","type":"string"},
    {"name":"question","source":"input.question","type":"string"}
  ]}',
  2,
  (SELECT DataSourceId FROM DataSource WHERE Alias='sandbox-deepseek-llm'),
  1
);
```

💡 **Qué dice esto sin que sepas SQL:** *“Voy a registrar en la base de datos UNA habilidad de Mika llamada `hr.summarize_policy`. La habilidad incluye: las instrucciones que le voy a dar al AI (el `systemPrompt`), una plantilla del mensaje del usuario con huecos para el documento y la pregunta, qué modelo usar por defecto, cuántos tokens máximo gastar, y qué hacer si el modelo primario falla (caer a OpenRouter).”*

“Si mañana quiero cambiar el system prompt, NO redeploy la app. Hago UPDATE a esta fila y el siguiente request usa el prompt nuevo. Si quiero rollback al prompt viejo, tengo el historial en la tabla `OperationHistory` (versionamiento built-in, vía triggers SQL).”

Esto es la diferencia entre vibe-coding y AI engineering. El vibe-coder edita el prompt en un archivo, hace commit, redeploy, espera 8 minutos a que pase CI, y reza. El AI engineer hace `UPDATE Operation SET TargetSpec = ... WHERE OperationKey = ...` y el siguiente request usa el nuevo prompt **con auditoría built-in**.

🟢 **Objetivo AI-102 aprobado en Mika:** *Implementación de generative AI con prompts versionados y catalogizables.*

8.2 — Multi-proveedor: diez músicos, un director

Objetivo AI-102: *“Manage Azure AI services and select the right model for the task.”*

El sesgo del examen: AI-102 asume que vas a usar Azure OpenAI Service. Las preguntas dicen “Azure OpenAI” en cada esquina. Es comprensible — es la plataforma de Microsoft.

Lo que decidí hacer diferente: Honrar la idea (separar el llamador del proveedor) pero **NO atarme solo a Azure OpenAI**. Mika habla con diez proveedores distintos detrás de la misma interfaz.

Esto vive en `api/llm_runner.py`, líneas 102-113:

```
# api/llm_runner.py
_PROVIDER_FACTORIES: dict[str, type[ILLMProvider]] = {
    "Groq":      LlamaProvider,      # Llama via Groq (ultra-fast)
    "DeepSeek": DeepSeekProvider,    # DeepSeek directo
    "Gemini":    GeminiProvider,     # Google Gemini
    "Mistral":   MistralProvider,    # Mistral (Francia)
    "Anthropic": AnthropicProvider,  # Claude
    "OpenAI":    OpenAIProvider,     # GPT vanilla
    "OpenRouter": OpenRouterProvider, # Agregador de muchos
    "Qwen":      QwenProvider,       # Alibaba
    "Glm":       GlmProvider,        # Tsinghua
    "Kimi":      KimiProvider,       # Moonshot
}
```

💡 **Qué dice este código:** “Tengo un diccionario donde la llave es el NOMBRE del proveedor (un string corto) y el valor es la CLASE que sabe hablar con ese proveedor. Cuando llega una operación que dice ‘usa Anthropic’, voy a este diccionario, encuentro `AnthropicProvider`, y la instancio.”

“Si mañana quiero agregar el proveedor #11 — por decir Cohere — agrego una clase `CohereProvider`, la mapeo en este diccionario, agrego una fila a la tabla `DataSource`, y ya. No tengo que modificar el resto de la lógica.”

Todas esas clases implementan la **misma interfaz** (`ILLMProvider`), que es básicamente un contrato que dice: “todos los músicos saben hacer `generate(messages, max_tokens, temperature)` y todos devuelven la misma estructura `LLMResponse`.”

```
# providers/illm_provider.py (extracto)
class ILLMProvider(Protocol):
    name: str

    async def generate(
        self,
        messages: list[LLMMessage],
        *,
        max_output_tokens: int = 1024,
        temperature: float = 0.2,
    ) -> LLMResponse: ...
```

💡 **Qué dice esto:** “Cualquier proveedor que quiera entrar a la orquesta tiene que saber tocar la misma canción base: recibe una lista de mensajes (system + user), tiene un límite de tokens, una temperatura (qué tan creativo), y devuelve la respuesta con metadata (texto, tokens usados, modelo final, finish_reason). Si un proveedor nuevo no cumple esta interfaz, no entra. Punto.”

Por qué esto importa más allá del examen:

Pregunta del CTO	Si solo usas Azure OpenAI	Lo que Mika contesta
¿Y si Azure OpenAI sube precio 40%?	Renegocias o aguantas	Cambias DataSource a DeepSeek (60% más barato) — 1 fila SQL
¿Y si un cliente exige que no usen OpenAI por compliance?	Refactor grande	Cambias DataSource a Mistral o Anthropic — 1 fila SQL
¿Y si quieren probar Claude vs GPT-4 para una operación específica?	Imposible sin A/B custom	Clonar la Operation, cambiar el DataSourceId , comparar audit rows

● **Objetivo AI-102 aprobado en Mika** (y extendido): El examen te examina manejar 1 proveedor bien. Mika maneja 10 detrás de una abstracción común — superando lo que el examen exige.

8.3 — Failover: cuando un músico falla

Objetivo AI-102: “Ensure reliability of the generative AI solution.”

Este es el área donde MÁS me alejé del examen oficial, por la razón que conté en la sección 7. Microsoft dice “deploya en otra región”; Mika dice “deploya con otro vendor”.

La lógica vive en el método `_is_failover_eligible` y en el loop de `_run_async` en `llm_runner.py` . Te muestro el corazón:

```
# api/llm_runner.py - la decisión de failover
def _is_failover_eligible(err: BaseException) -> bool:
    """Decide si un error amerita failover.

    Eligible:
    - ProviderError marcado retryable=True (rate limits, timeouts, 5xx)
    - Errores no-retryable pero con código de quota/payment (402)
      que indica cap del vendor agotado
    - 404 not_found (modelo deprecated/delisted en este vendor)

    NOT eligible:
    - InvalidPromptTemplate / InvalidLlmReturnShape (logic bug)
    - UnsupportedProvider (config bug)
    - Auth errors (401) - típicamente la key se dañó
    """
    if isinstance(err, ProviderError):
        if err.retryable:
            return True
        msg = str(err).lower()
        failover_signals = (
            "quota", "402", "payment", "insufficient",
            "rate", "429", "503", "504", "timeout", "unavailable",
            "404", "not_found", "deprecated", "decommissioned",
        )
        if any(sig in msg for sig in failover_signals):
            return True
    return False
```

💡 Qué dice este código en lenguaje de calle:

"Cuando un proveedor falla, NO automáticamente brinco al siguiente. Primero pregunto: ¿de qué tipo es la falla?

Si la falla es 'el proveedor está caído', 'subió rate limit', 'agotaste cuota', ENTONCES sí — intenta con el siguiente proveedor. El problema no soy yo (Mika), es el músico.

Si la falla es 'tu prompt está mal formateado', 'pediste un retorno que no soporto', 'tu API key está chueca', NO BRINCO. ¿Por qué? Porque ese mismo prompt mal formateado va a fallar igual con el siguiente proveedor. Brincar a un fallback ahí sería esconder el bug, no resolverlo. El error tiene que llegar arriba para que el ingeniero (yo) lo arregle."

Y luego está la **lista de attempts**, que es la coreografía del failover. Vive en `_run_async` :


```
# api/llm_runner.py - la cadena de attempts
primary_attempt = {
    "label": "primary",
    "provider": resolved.data_source_provider,
    "connectionSecretName": resolved.connection_secret_name,
    "modelHint": spec.get("modelHint"),
}
attempts = [primary_attempt]
for i, fb in enumerate(fallbacks):
    attempts.append({
        "label": f"fallback{i}",
        "provider": fb.get("provider"),
        "connectionSecretName": fb.get("connectionSecretName"),
        "modelHint": fb.get("modelHint"),
    })

# Luego para cada attempt, intenta. Si falla y es failover-eligible, sigue.
# Cada attempt deja huella en routing_path para auditoría.
routing_path: list[str] = []
for attempt in attempts:
    route_marker = f"{attempt['label']}:{attempt['provider']}:{attempt['modelHint']}"
    try:
        # ... intenta el llamado ...
        routing_path.append(f"{route_marker}:ok")
        return success
    except BaseException as e:
        routing_path.append(f"{route_marker}:failed-{type(e).__name__}")
        if not _is_failover_eligible(e):
            raise # bug verdadero, no esconderlo
        continue # intenta el siguiente
```

💡 Qué dice esto:

“Construyo la lista de músicos que voy a intentar. El primero es el primario (definido en `Operation.DataSourceId`). Después siguen los fallbacks (definidos en `TargetSpec.fallbacks`). Voy uno por uno. Si el músico responde, anoto `:ok` en el camino, devuelvo la respuesta, y termino. Si el músico falla y la falla es ‘no es mi culpa, es del músico’, anoto `:failed-RateLimitError` y paso al siguiente. Si la falla es ‘es bug mío’, anoto `:fatal-InvalidPromptTemplate` y RE-LANZO el error. No intento esconder el bug brincándolo.”

El `routing_path` se guarda en `ExecutionLog`. Eso significa que en 6 meses puedes hacer un query como:

```
SELECT
    COUNT(*) AS times_primary_failed,
    SUBSTRING(RoutingPath, 1, 80) AS sample
FROM ExecutionLog
WHERE RoutingPath LIKE 'primary:Anthropic%failed%'
    AND CreatedUtc > DATEADD(month, -1, GETUTCDATE())
GROUP BY SUBSTRING(RoutingPath, 1, 80)
ORDER BY times_primary_failed DESC;
```

...y saber **EXACTAMENTE** cuántas veces Anthropic falló este mes, qué tipo de error fue, y a qué fallback caímos. Eso es **observabilidad real de failover, en SQL, sin Datadog ni Splunk**.

● **Objetivo AI-102 aprobado en Mika** (y extendido a multi-vendor): El examen pregunta failover dentro de Azure. Mika hace failover entre vendors completamente distintos — con auditoría grade para mostrarlo al cliente.

8.4 — Key Vault: la cert dice “no hardcodees API keys”

Objetivo AI-102: “Manage credentials and access to AI services securely.”

Una de las preguntas del examen es algo como: “Where should you store the API key for an Azure OpenAI deployment?” Y la respuesta correcta NO es “.env file” ni “App Settings” — es **Azure Key Vault con Managed Identity**.

Es decir: el examen literalmente dice “si tu API key está en un archivo de configuración, estás mal”.

Y aquí es donde 9 de cada 10 productos AI que veo en GitHub tienen su `OPENAI_API_KEY=sk-...` en un `.env` chequeado en `.gitignore` esperando que nunca se les escape. **Eso reprobaría AI-102**.

Lo que hice en Mika vive en `providers/keyvault.py` :

```
# providers/keyvault.py (núcleo del resolver)
from azure.identity.aio import DefaultAzureCredential
from azure.keyvault.secrets.aio import SecretClient

class KeyVaultSecretResolver:
    def __init__(self, *, vault_url: str) -> None:
        if not vault_url:
            raise ValueError("vault_url is required")
        self._vault_url = vault_url.rstrip("/") + "/"
        self._credential = DefaultAzureCredential()
        self._client = SecretClient(
            vault_url=self._vault_url,
            credential=self._credential,
        )

    async def get_secret(self, name: str, *, version: str | None = None) -> str:
        secret = await self._client.get_secret(name, version=version)
        return secret.value or ""
```

💡 Qué dice este código sin que sepas Python:

"Cuando arranca el resolver, le pides la URL del Key Vault (la caja fuerte). Le dices: 'usa la credencial por defecto' — que es como decir 'identifícate con tu credencial corporativa, sea cual sea: si es producción usa la Managed Identity del App Service; si es local usa el `az login` que ya hice'.

Cuando alguien pide un secreto por nombre (ej: `mika-anthropic-api-key`), va a la caja fuerte, lo trae, lo devuelve, y se olvida. No lo cachea agresivamente, no lo escribe en log, no lo expone en error messages."

El detalle fino que importa: `DefaultAzureCredential`. Esto es lo que AI-102 te examina porque es el patrón canónico de **una sola línea de código que funciona igual en local y en producción.**

- **En tu laptop:** usa el `az login` que hiciste. No necesitas API key.
- **En Azure App Service / Container Apps:** usa la **Managed Identity** asignada al recurso. NO HAY API key en NINGÚN archivo. El recurso de Azure se identifica con su propia identidad corporativa.

💡 **Por qué importa:** *"Cuando un developer del equipo nuevo entra, no le tengo que mandar API keys por Slack ni por email. Le doy permiso de 'Key Vault Secrets User' a SU cuenta de Azure AD y listo — al hacer `az login` con su cuenta, su laptop puede leer secretos. Si se va de la empresa, le quito el rol y SU laptop deja de poder leer secretos al instante. Nunca hubo una API key que rotar."*

"Eso se llama 'secret hygiene' y es uno de los grandes diferenciadores entre un sistema profesional y un sistema vibe-codeado."

La conexión con el LLM runner: Cuando `llm_runner.py` necesita la API key de Anthropic, NO lee `.env`. Hace esto:

```
# api/llm_runner.py - uso del resolver
resolver = KeyVaultSecretResolver(vault_url=keyvault_url)
api_key = await resolver.get_secret(secret_name)
provider = AnthropicProvider(api_key=api_key, model=model_hint)
```

🟢 **Objetivo AI-102 aprobado en Mika** (con *Managed Identity production-ready*): Cero API keys hardcoded. Cero secretos en archivos. Acceso por identidad, no por string en `.env`.

8.5 — Métricas LLM: el “cuaderno de Tesorería” para AI

Objetivo AI-102: “Monitor cost and consumption of generative AI solutions.”

El examen pregunta cómo monitoreas tokens y costos. La respuesta oficial es “usa Azure Monitor + Cost Management”. Eso está bien — para Azure. Pero **Mika usa 10 proveedores, no solo Azure OpenAI**. Así que tenía que tener mi propia “tabla de tokens” para que el reporte de costo sea unificado.

Eso vive en `ExecutionLog`, en la migración `024_executionlog_llm_columns.sql`:

```
-- db/024_executionlog_llm_columns.sql
ALTER TABLE dbo.ExecutionLog
    ADD LlmProvider      NVARCHAR(50) NULL,
        LlmModel        NVARCHAR(100) NULL,
        LlmInputTokens  INT           NULL,
        LlmOutputTokens INT           NULL;
```

💡 **Qué dice esto:** “A la tabla ‘cuaderno de Tesorería’ (que ya tenía: fecha, status, correlation id, qué operación), le agregué cuatro columnas dedicadas a LLM: qué proveedor se usó, qué modelo específico, cuántos tokens de entrada (lo que mandamos), cuántos de salida (lo que respondió).”

Cómo se popula: El `executor.py` recibe el `LlmRunOutcome` que devuelve `llm_runner.py`, y lo escribe en la fila correspondiente:

```
# Conceptualmente, en el branch LLM del executor
outcome = await run_llm_operation_async(resolved, bound, keyvault_url=kv_url)
# outcome.input_tokens, outcome.output_tokens, outcome.provider_name, outcome.model

await log_execution_finish(
    execution_log_id=execution_log_id,
    status='ok',
    llm_provider=outcome.provider_name,
    llm_model=outcome.model,
    llm_input_tokens=outcome.input_tokens,
    llm_output_tokens=outcome.output_tokens,
    # ... etc ...
)
```

Y el resultado es que puedo hacer queries de costo brutales como:

```
-- Cost per provider, last 30 days
SELECT
    LlmProvider,
    COUNT(*) AS calls,
    SUM(LlmInputTokens) AS input_tokens,
    SUM(LlmOutputTokens) AS output_tokens,
    SUM(LlmInputTokens + LlmOutputTokens) AS total_tokens
FROM ExecutionLog
WHERE LlmProvider IS NOT NULL
    AND CreatedUtc > DATEADD(day, -30, GETUTCDATE())
GROUP BY LlmProvider
ORDER BY total_tokens DESC;
```

💡 Qué dice este query sin que sepas SQL:

"Dame, de los últimos 30 días, cuántas llamadas hice por proveedor, cuántos tokens de entrada, cuántos de salida, y total. Ordenado de mayor a menor."

"En el día de ayer (14 jun) corrí algo parecido y el reporte real fue: Anthropic = 70 calls, ~\$1.20 USD. DeepSeek + Gemini + GLM + Qwen + Mistral combinados = ~\$0.10 USD. Total del día completo de desarrollo intensivo: \$1.30 USD. El runway de Mika a este ritmo se mide en años, no meses."

El detalle no obvio: en la mayoría de productos AI que veo en GitHub, el reporte de costo **no existe**. La gente sabe cuánto le cuesta Azure OpenAI porque revisa la factura mensual. **Pero no sabe cuánto le cuesta CADA OPERACIÓN.** No puede decirle al cliente "esta consulta te costó 0.03 centavos". En Mika, esa cifra está en SQL, fila por fila, desde el primer commit.

● **Objetivo AI-102 aprobado en Mika:** Métricas de consumo capturadas en cada llamada, sin depender de Azure Monitor exclusivamente.

8.6 — Prompt engineering catalogable

Objetivo AI-102: *"Implement prompt engineering techniques (system messages, few-shot, structured output)."*

En Mika, el prompt se ensambla en el método `_build_messages` :

```
# api/llm_runner.py
def _build_messages(target_spec: dict, bound: dict[str, Any]) -> list[LLMMessage]:
    template = target_spec.get("userPromptTemplate")
    if not template:
        raise InvalidPromptTemplate(
            "TargetSpec.userPromptTemplate is required for llm_call operations."
        )

    # Stringify cualquier valor no-string (int, Decimal, datetime)
    # para que el .format() de Python interpole limpio
    formattable = {
        k: (v if isinstance(v, str) else (str(v) if v is not None else ""))
        for k, v in bound.items()
    }

    try:
        user = template.format(**formattable)
    except KeyError as e:
        slot = e.args[0] if e.args else "?"
        raise InvalidPromptTemplate(
            f"userPromptTemplate references {{{slot}}} but no ParameterMap "
            f"parameter with name '{slot}' was bound."
        ) from e

    msgs: list[LLMMessage] = []
    system = target_spec.get("systemPrompt")
    if system:
        msgs.append(LLMMessage(role="system", content=system))
    msgs.append(LLMMessage(role="user", content=user))
    return msgs
```

💡 Qué dice este código:

"Lee la plantilla del prompt del catálogo (`userPromptTemplate`). Tomas los inputs que llegaron al request (`bound`) y rellenas los huecos de la plantilla. Si el prompt dice `Document: {document}` y nadie te mandó `document` , falla con un error CLARO diciendo qué slot faltó. Si todo está bien, arma el mensaje: primero el rol 'system' con las instrucciones, luego el rol 'user' con el contenido lleno."

Lo que es importante AI-102:

1. `role: system` + `role: user` es exactamente el patrón canónico de OpenAI Chat Completions API que el examen examina.
2. **La plantilla con slots** `{document}` `{question}` es la versión catalogizable del *prompt template engineering* — no estás escribiendo el prompt en cada llamada, lo declaraste UNA vez en SQL.
3. **El error explícito cuando falta un slot** evita el bug clásico del vibe-coder: el prompt llega con un `{document}` literal sin reemplazar porque alguien olvidó mandar el campo. En Mika eso es un error fatal con mensaje accionable; en vibe-code es un response confuso del AI que dice "no sé de qué hablas".

🟢 **Objetivo AI-102 aprobado en Mika:** Prompt engineering catalogizado, con validación de slots faltantes y errores accionables.

8.7 — Structured output: las 4 formas de proyectar respuestas

Objetivo AI-102: "Implement structured output from generative AI models."

El examen te pregunta cómo manejar la respuesta del modelo: ¿solo texto? ¿JSON? ¿Solo metadata? ¿Stream?

Mika tiene **4 ReturnShapes** y la lógica vive en `_project_llm_response` :

```
# api/llm_runner.py
def _project_llm_response(resp: LLMResponse, return_shape: int) -> Any:
    if return_shape == 5:                # void
        return None
    if return_shape == 6:                # stream (reserved)
        raise InvalidLlmReturnShape(
            "ReturnShape 6 (stream) is reserved; LLM streaming not wired yet."
        )
    if return_shape == 1:                # scalar text
        return resp.text
    if return_shape == 2:                # JSON envelope (default)
        return {
            "text": resp.text,
            "model": resp.model,
            "input_tokens": resp.input_tokens,
            "output_tokens": resp.output_tokens or 0,
            "finish_reason": resp.finish_reason,
        }
    raise InvalidLlmReturnShape(
        f"ReturnShape {return_shape} is not valid for llm_call operations; "
        f"use 1, 2, 5, or 6."
    )
```

💡 Qué dice esto:

"Cada Operation declara qué tipo de respuesta espera. Hay 4 formas:

- `ReturnShape = 1` → 'solo dame el texto, sin metadata'. Útil cuando el caller solo quiere la cadena.
- `ReturnShape = 2` → 'dame el texto envuelto con metadata (modelo, tokens, finish_reason)'. Default, útil cuando el caller quiere auditar.
- `ReturnShape = 5` → 'no me devuelvas nada, solo dispara y olvida'. Útil para operaciones tipo 'log this' o 'trigger background job'.
- `ReturnShape = 6` → 'stream' — reservado para cuando wireemos streaming."

"Si una Operation pide ReturnShape inválido (ej: 3), falla explícito con error claro. Nada de devolver 'undefined' como JavaScript vibe-codeado."

🟢 **Objetivo AI-102 aprobado en Mika:** Structured output declarado per-Operation, validado, sin sorpresas en el shape de la respuesta.

9. Recap y puente a Parte 3

Lo que cubrió esta parte

Si llegaste hasta aquí, ya viste **cómo cada uno de los grandes objetivos de AI-102 se materializa en archivos reales** dentro de `mika-core`:

1. ☒ Prompts catalogados en SQL (no hardcoded)
2. ☒ Multi-proveedor con interfaz común (10 músicos vs 1)
3. ☒ Failover multi-vendor con clasificación de errores
4. ☒ Key Vault con Managed Identity (cero API keys en archivos)
5. ☒ Métricas de consumo y costo por llamada en SQL
6. ☒ Prompt engineering catalogizable con validación de slots
7. ☒ Structured output declarado y validado por Operation

Lo que NO cubrí (y queda honesto declararlo)

🚧 Cosas que AI-102 examina pero Mika todavía no implementa fully:

- **Content filtering nivel Azure** (Azure AI Content Safety) — Mika depende del content filter built-in de cada proveedor. No tiene capa propia de filtrado.
- **Computer Vision end-to-end** — solo OCR parcial en MikaRH. No image classification, no object detection.
- **Streaming output** (`ReturnShape = 6`) — declarado pero no wireado en `_project_llm_response`. El streaming del Builder lo hace `/chat/scaffold/stream` con un parser de markers custom, no via `ReturnShape`.
- **Groundedness tests en CI** — declarado en el post #1 pero todavía es backlog real, no tests verdes corriendo.

No declarar estos huecos sería deshonesto. Y “honest” es el modificador que más uso en este proyecto — porque la diferencia entre Mika y un wrapper de ChatGPT no es solo el código; es la cultura de **declarar lo que NO funciona** además de lo que sí.

Lo que viene en la Parte 3

Si en Parte 2 hablamos del **cerebro IA** (cómo Mika decide qué modelo usar y cómo procesa prompts), en Parte 3 vamos a hablar de:

- **AZ-204 — Cómo Mika CORRE:** FastAPI, SQL Server, mensajería, observabilidad, deployment. La cert “Azure Developer Associate” que te enseña cómo se construye un backend serio.

- **AZ-500 — Por qué Mika es SEGURA:** Identity, RBAC, network, audit log, threat modeling. La cert “Azure Security Engineer Associate” — la que más vibe se le ve en cómo está armado este sistema.
- **La comparativa with/without Azure:** si tomara Mika y la deployara sin Azure, ¿qué exactamente se rompe?

Esa es la Parte 3. Continúa en el mismo documento si lees todo de corrido, o en el archivo `mika-parte-3-az204-az500.md` si lo abres independiente.

Última actualización: 2026-06-15 06:00 CDMX · Parte 2 de 4 Código referenciado: `mika-core/api/llm_runner.py`, `mika-core/providers/keyvault.py`, `mika-core/db/008_llm_seed.sql`, `mika-core/db/024_executionlog_llm_columns.sql`

PARTE 3 — AZ-204 y AZ-500: cómo Mika corre y por qué es segura

Las dos certs que cierran la historia: una de developer, una de security engineer

Esta parte explica las dos certs restantes que dieron forma a Mika.

AZ-204 (Azure Developer Associate) responde “¿cómo corre Mika en producción?” — cómo se hostea, cómo guarda datos, cómo se monitorea, cómo se comunica con otros sistemas.

AZ-500 (Azure Security Engineer Associate) responde “¿por qué se le ve el flavor de seguridad?” — cómo se manejan identidades, cómo se protegen secretos, cómo se audita TODO, y por qué la seguridad no es un módulo aparte sino una propiedad transversal del sistema.

Al final hago una **comparativa concreta**: si tomara Mika y la deployara sin Azure, ¿qué exactamente se rompe? El answer no es “todo” — es algo más interesante.

10. AZ-204 en lenguaje claro

¿Qué es AZ-204, sin marketing?

AZ-204 es la cert oficial “**Azure Developer Associate**”. Si AI-102 te enseña a hablarle a los modelos, **AZ-204 te enseña a construir el resto del backend** que rodea a esos modelos: cómo se hostea, cómo guarda datos, cómo se monitorea, cómo se conecta con otros sistemas.

En cristiano: **es la cert que dice “esta persona sabe construir aplicaciones que viven en Azure y no se caen el viernes a las 5pm”**.

Pasé este examen en 2024. Lo recuerdo porque fue el primer examen donde me di cuenta de que **la diferencia entre un developer junior y un developer senior es el 80% sobre operaciones, no sobre código**. El código nuevo lo escribes en una semana; aprender a no romper lo que ya hay y a recuperarte cuando se rompe **es de toda la vida**.

Los cuatro grandes capítulos

Capítulo del examen	En lenguaje de calle
Develop Azure compute solutions	Cómo corres tu app: App Service, Container Apps, Functions, VMs
Develop for Azure storage	Dónde guardas datos: Cosmos DB, Blob Storage, SQL, Tables
Implement Azure security	Cómo manejas auth y secrets DESDE EL CÓDIGO (overlap con AZ-500)
Monitor, troubleshoot, optimize	Cómo te enteras cuando algo falla en producción
Connect to & consume services	Cómo Mika habla con APIs, queues, event grid, otros sistemas

Mika toca **todos** estos capítulos. Cada uno tiene un mapeo concreto a un archivo del repo.

11. Cómo apliqué AZ-204 en mika-core

11.1 — Compute: FastAPI + ASGI, deployable en cualquier compute target

Objetivo AZ-204: *"Develop solutions that use Azure App Service, Functions, or Container Apps."*

Mika es una app **FastAPI**. FastAPI corre encima de `uvicorn` (servidor ASGI). Eso significa que puedo deployar la misma imagen en:

- **Azure App Service for Linux** (el más simple — sube código + start command)
- **Azure Container Apps** (containerizado, escala a cero cuando no hay tráfico)
- **Azure Kubernetes Service** (overkill para hoy, pero opción si crece)
- **Azure Functions** (no muy buen fit aquí, FastAPI no es serverless-native)

Lo que importa es **no atarse a uno**. La app entry point vive en `api/main.py` :

```
# api/main.py
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    # bootstrap secrets from KV here if needed
    yield

app = FastAPI(lifespan=lifespan)

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}

@app.post("/operations/{operation_key}/execute")
async def execute(operation_key: str, request: ExecuteRequest) -> ExecuteResponse:
    # ... bind params, run executor, return result
```

💡 Qué dice esto sin que sepas Python:

"FastAPI es como Express en Node.js: defines endpoints (`/health` , `/operations/.../execute`) y cuando llega un request, ejecuta la función correspondiente. El `lifespan` es un mecanismo para hacer setup/teardown ordenado: bootstrap de secrets al arranque, cleanup de conexiones al apagar. Eso es importante para que App Service haga health checks bien y para que recargos calientes no dejen recursos colgados."

🟢 **Objetivo AZ-204 aprobado en Mika:** App stateless, lifecycle bien manejado, deployable en cualquier compute target de Azure.

11.2 — Storage: SQL Server como fuente de verdad (con MARS habilitado)

Objetivo AZ-204: "Develop solutions that use Azure SQL Database."

Esta decisión es la **más importante de la arquitectura** y la cuento aquí porque viene del examen tanto como de la experiencia.

El examen pregunta: "Where should you store metadata for a multi-tenant SaaS?" y las opciones suelen incluir Cosmos DB, Table Storage, SQL. La respuesta típica de un junior es Cosmos DB ("¡es NoSQL, escala!"). La respuesta de un senior es **SQL** — porque relaciones, triggers, transacciones y auditoría built-in son **lo que un sistema enterprise necesita**, y SQL tiene años de tooling encima.

En Mika **todo** vive en SQL Server (Azure SQL para producción, SQL Server local para desarrollo):

- Catalog (`Operation` , `DataSource` , `ParameterMap` , `Principal`)
- Audit (`ExecutionLog` , `OperationHistory` con triggers)
- Sesiones de chat (en-memory por ahora — backlog mover a tabla)

- Documentos de proceso (MikaRH)

La conexión vive en `api/main.py` :

```
# api/main.py
def _connection_string() -> str:
    s = settings()
    return (
        "DRIVER={ODBC Driver 17 for SQL Server};"
        f"SERVER={s.sql_server};"
        f"DATABASE={s.sql_database};"
        f"UID={s.sql_user};"
        f"PWD={s.sql_password};"
        "Encrypt=yes;TrustServerCertificate=no;Connection Timeout=10;"
        "MARS_Connection=Yes;"
    )

_SESSION_SETUP = (
    "SET ANSI_NULLS ON; "
    "SET QUOTED_IDENTIFIER ON; "
    "SET ANSI_PADDING ON; "
    "SET ANSI_WARNINGS ON; "
    "SET CONCAT_NULL_YIELDS_NULL ON; "
    "SET ARITHABORT ON; "
    "SET NUMERIC_ROUNDABORT OFF;"
)

def _open_connection() -> pyodbc.Connection:
    conn = pyodbc.connect(_connection_string(), autocommit=True)
    conn.cursor().execute(_SESSION_SETUP)
    return conn
```

💡 Qué dicen estos detalles que parecen aburridos:

- `Encrypt=yes;TrustServerCertificate=no` — el tráfico SQL viaja **encriptado**, y NO aceptamos certificados auto-firmados. AZ-204 examina esto literal: "is your connection string secure?".
- `MARS_Connection=Yes` — *Multiple Active Result Sets*. Esto permite que el executor lea resultados Y escriba audit row al mismo tiempo sin tener que abrir una segunda conexión. **Pequeño detalle, gran impacto en throughput.**
- El `_SESSION_SETUP` con `SET QUOTED_IDENTIFIER ON` — REQUERIDO para que funcionen filtered indexes e indexed views en Azure SQL. ODBC Driver 17 lo deja OFF por defecto. Esto NO lo aprendes leyendo docs — lo aprendes cuando un query rompe en producción y no sabes por qué.

🟢 **Objetivo AZ-204 aprobado en Mika:** Conexión SQL encriptada, hardened, con session setup explícito para evitar bugs sutiles de Azure SQL.

11.3 — Audit logging: monitoring built-in, no Application Insights opcional

Objetivo AZ-204: “Implement logging and monitoring for Azure solutions.”

El examen te dice: usa **Application Insights**. Eso está bien, y Mika lo va a usar cuando esté en producción.

PERO — y este es un PERO importante — Application Insights es **monitoring de la app**: latencia, errores, dependencias, exceptions. NO es monitoring del **negocio**: qué operación se invocó, con qué inputs, qué tokens gastó, qué CorrelationId estampó.

Mika tiene su propia capa de business audit ([ExecutionLog](#)) que es **mucho más profunda** que App Insights para este dominio. Cada llamada deja una fila con:

```
-- Las columnas de ExecutionLog que importan para AZ-204 monitoring
ExecutionLogId      BIGINT IDENTITY,
CorrelationId       UNIQUEIDENTIFIER,    -- trazable end-to-end
OperationId         INT,                  -- qué operación se invocó
PrincipalId         INT,                  -- quién la invocó (RBAC)
Status              NVARCHAR(20),        -- ok | error | timeout
StartedUtc          DATETIME2(7),
FinishedUtc         DATETIME2(7),
DurationMs          AS DATEDIFF(MILLISECOND, StartedUtc, FinishedUtc),
ErrorMessage        NVARCHAR(MAX),       -- truncado a 500 chars en runner
ResultRowCount      INT,
InvokedVia          CHAR(3),              -- api | mcp | job | cli
LlmProvider         NVARCHAR(50),        -- AI-102 cross-over
LlmModel            NVARCHAR(100),
LlmInputTokens      INT,
LlmOutputTokens     INT,
RoutingPath         NVARCHAR(2000)      -- ej: 'primary:Anthropic:claude-sonnet-4-6:ok'
```

Cuando un cliente me pregunta “¿cuántas operaciones corren al día?” o “¿cuál es el provider más caro este mes?” o “¿qué Principal está consumiendo más?” — la respuesta es un query SQL de 10 líneas. **No tienes que tener Kusto, ni Splunk, ni instrumentar nada nuevo.** El monitoring del negocio nace con el sistema.

💡 Por qué importa más allá del examen:

App Insights es genial para ‘mi API tardó 2.5 segundos’. Pero un cliente serio te pregunta cosas como ‘cuántas veces alguien intentó ejecutar `hr.summarize_policy` con un documento de más de 50KB en el último mes’ — y para eso necesitas datos del DOMINIO, no del transporte. Por eso ExecutionLog es propio.

🟢 **Objetivo AZ-204 aprobado en Mika:** Business-level monitoring built-in en SQL, independiente de App Insights (que se agregará sobre — no en lugar de — esto).

11.4 — Connect & consume services: el MCP server

Objetivo AZ-204: “Connect to and consume Azure and third-party services.”

Mika expone sus Operations **dos veces**:

1. Como **REST API** (típico FastAPI) — para apps web, móvil, scripts curl.
2. Como **MCP server** (Model Context Protocol) — para que herramientas tipo Claude Code, Cursor, ChatGPT con MCP se conecten directo y llamen las Operations como si fueran herramientas nativas.

El MCP server vive en `api/mcp_server.py`. Para el examen, lo relevante es que **una sola Operation en el catálogo se expone por DOS canales sin código duplicado**. La lógica de bind + executor + audit es la misma. Solo cambia la cáscara de entrada/salida.

```
# Conceptualmente, lo que MCP hace
@mcp.tool()
async def hr_summarize_policy(document: str, question: str) -> str:
    # Mismo executor que el REST endpoint
    resolved = resolve_operation('hr.summarize_policy')
    result = await execute_operation(resolved, {"document": document, "question": question})
    return result.body
```

💡 Qué significa esto en práctica:

“Cuando un developer del cliente usa Cursor o Claude Code y le pregunta a SU IDE ‘resúmeme esta política’, el IDE puede usar la Operation de Mika sin abrir el navegador. Es una integración nativa. Eso convierte a Mika de ‘una app más’ a ‘una extensión del IDE del cliente’ — uno de los moats más fuertes para B2B LATAM tech.”

● **Objetivo AZ-204 aprobado en Mika (y extendido):** Una sola lógica, dos canales de exposición (REST + MCP). Connect & consume no es solo “consumir Azure Services” — es **ser** un servicio consumible.

12. AZ-500 en lenguaje claro

¿Qué es AZ-500, sin marketing?

AZ-500 es “**Azure Security Engineer Associate**”. Si AI-102 te enseña a hablar con Als y AZ-204 te enseña a construir backend, **AZ-500 te enseña a NO ser hackeado y a poder demostrarlo en una auditoría legal**.

Esta es **la cert con más vibe en Mika**. No es accidente. Cuando empecé a diseñar el sistema, escribí esta nota en engram el 2026-06-09 a medianoche:

"deberíamos meterle mucha seguridad, que se vea que pasé AZ-500"

Y de ahí en adelante, **CADA decisión de arquitectura** pasó por la pregunta: *"¿qué diría un AZ-500 al ver esto?"*

Los cuatro grandes dominios

Dominio del examen	En lenguaje de calle
Manage identity and access	Quién entra, con qué credencial, y qué puede hacer (Entra ID, Conditional Access, RBAC)
Secure networking	Cómo proteges la red (NSG, firewall, Private Endpoints, DDoS)
Secure compute, storage, databases	Cómo proteges el cómputo y los datos (TDE, Always Encrypted, Managed Identity)
Manage security operations	Cómo te enteras y respondes a amenazas (Defender, Sentinel, alerting)

La idea central que cambió mi forma de programar:

AZ-500 NO trata seguridad como un módulo aparte. Trata seguridad como una PROPIEDAD TRANSVERSAL del sistema.

En vibe-code, agregas "el módulo de auth" al final. En AZ-500-code, **cada línea pregunta**: ¿quién la ejecuta? ¿con qué identidad? ¿queda registrado? ¿qué pasa si el atacante controla el input?

Esa diferencia es la que distingue a un dev que pasó el examen del que solo lo memorizó.

13. Cómo apliqué AZ-500 en mika-core

13.1 — Identity & access: Principal table + RBAC custom

Objetivo AZ-500: *"Manage role-based access control (RBAC)."*

En la mayoría de apps que veo en GitHub, "auth" es un par de líneas que validan un JWT y meten un `user_id` en el contexto. Punto. No hay roles. No hay aprobación de cambios. No hay rotación.

En Mika, **la autorización vive en la tabla `Principal`**. Cada llamada al executor pasa un `PrincipalContext`:

```
# api/executor.py (conceptual)
@dataclass(slots=True)
class PrincipalContext:
    principal_id: int          # qué Principal está ejecutando
    invoked_via: str           # 'api' | 'mcp' | 'job' | 'cli'

# En cada execute:
def execute_operation(resolved: ResolvedOperation,
                     inputs: dict,
                     principal: PrincipalContext) -> ExecutionResult:
    # 1. Verifica que el Principal tenga permiso para esta Operation
    # 2. Estampa principal_id en ExecutionLog
    # 3. Si UsesTransaction=1, abre TX; si no, autocommit
    # 4. Ejecuta el SP o el LLM call
    # 5. Cierra con LogExecutionFinish + status
```

💡 Qué dice esto:

“Cada llamada NO viene anónima. Viene con un Principal identificable. Antes de ejecutar la Operation, Mika verifica que ese Principal tenga permiso para invocarla. Y el `principal_id` queda estampado en la fila de `ExecutionLog`. En 5 años, si alguien pregunta ‘quién ejecutó `mika.scaffold_skeleton` con este spec el 14 de junio’, la respuesta es `SELECT * FROM ExecutionLog WHERE ExecutionLogId = 981`.”

🟡 **Honest declaration (audit-honest):** Hoy el `PrincipalId=1` está hardcoded en `_DEFAULT_PRINCIPAL` mientras integramos Entra ID. **El mecanismo de Principal + RBAC está armado en la tabla;** lo que falta es la capa JWT/Entra para que el `principal_id` salga del token, no de un default. Está en el roadmap Q3 (sem 3-4).

Esto es exactamente el patrón AZ-500 — **estructura primero, integración después** — porque si la estructura está mal, el Entra ID no la arregla. Si la estructura está bien, Entra ID se inserta sin refactor.

🟢 **Objetivo AZ-500 aprobado en Mika** (*estructura — pendiente integración Entra*): RBAC scoped por Principal en la tabla, propagado a audit log, listo para Entra ID encima.

13.2 — Secure storage of secrets: Managed Identity end-to-end

Objetivo AZ-500: “Secure secrets using Azure Key Vault and Managed Identity.”

Esto ya lo cubrí en la **Parte 2 sección 8.4** desde el ángulo AI-102. Pero AZ-500 lo examina desde un ángulo más estricto. AI-102 dice “no hardcodees keys”. AZ-500 dice:

- **Maneja la identidad que accede al vault**, no la API key. Si la identidad se compromete, revocas; si la key se compromete, rotas en TODOS los lugares donde se usó.
- **Audita cada acceso al vault** — Azure Key Vault logs vía Diagnostic Settings hacia Log Analytics. Quién leyó qué secreto, cuándo.

- **Rota secretos con expiración** — todo secreto debe tener fecha de expiración y rotación automática programada.

En Mika el `DefaultAzureCredential` resuelve esto en el código. La parte de **diagnostic settings + rotación es infrastructure as code** (Bicep / Terraform) que vive fuera del repo de la app — y eso es CORRECTO. AZ-500 enseña que las políticas de KV NO viven en código de aplicación; viven en IaC.

💡 El detalle no obvio:

“En vibe-code, ves esto: `api_key = os.getenv('OPENAI_API_KEY')` . Si el `.env` se filtra, esa key flota en GitHub. Si rotas la key, tienes que cambiarla en cada deploy, cada local environment, cada CI runner.

En Mika, el secreto NUNCA toca el sistema de archivos. La identidad del App Service (la Managed Identity de Azure) tiene un rol `Key Vault Secrets User` sobre el vault. Cuando Mika pide el secreto, lo lee directo del vault con esa identidad. Si rotas el secreto, el siguiente request ya usa el nuevo valor. No hay que tocar nada más. Eso es continuous secret hygiene — uno de los conceptos que AZ-500 más empuja.”

● **Objetivo AZ-500 aprobado en Mika:** Identity-based access al vault, secretos sin tocar disco, rotación-friendly por diseño.

13.3 — Audit log inmutable: el “cuaderno de Tesorería” como control de seguridad

Objetivo AZ-500: “Manage security operations using SIEM tools and audit trails.”

En la Parte 1 usé la metáfora del “cuaderno de Tesorería” para `ExecutionLog` . En la Parte 2 mostré las columnas LLM. Acá lo cuento desde el ángulo AZ-500 — porque **ExecutionLog no es solo observabilidad; es un control de seguridad.**

Lo que la hace un control de seguridad:

1. **Inmutabilidad de escritura** — `ExecutionLog` NO tiene `UPDATE` en su API. Solo `INSERT` . La fila se escribe una vez al inicio (LogExecutionStart) y se “cierra” con un INSERT a tabla de finish vinculada. **No hay forma de modificar una fila después de cerrada.**
2. **CorrelationId trazable end-to-end** — el mismo `UNIQUEIDENTIFIER` estampado en la fila de audit aparece en CADA archivo generado, CADA email enviado, CADA respuesta al cliente. Si en 6 meses hay disputa “este reporte está mal”, buscas por CorrelationId y reconstruyes EXACTAMENTE qué pasó.
3. **OperationHistory con triggers SQL** — cuando alguien cambia el `TargetSpec` de una Operation (ej: el prompt), un trigger en SQL Server escribe la fila vieja a `OperationHistory` con `ValidFrom/ValidTo` . **Versionado temporal automático, NO depende de que el dev se acuerde de hacerlo.**

Aquí está el patrón en SQL del trigger (de `db/037_operation_versioning.sql`):

```
-- db/037_operation_versioning.sql (extracto conceptual)
CREATE TRIGGER trg_Operation_VersionOnChange
ON dbo.Operation
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.OperationHistory (
        OperationId, OperationKey, TargetSpec, ParameterMap,
        ReturnShape, DataSourceId,
        ValidFrom, ValidTo, ChangedBy
    )
    SELECT
        d.OperationId, d.OperationKey, d.TargetSpec, d.ParameterMap,
        d.ReturnShape, d.DataSourceId,
        d.LastModifiedUtc, SYSUTCDATETIME(),
        SUSER_SNAME()
    FROM deleted d
    JOIN inserted i ON i.OperationId = d.OperationId
    WHERE
        ISNULL(d.TargetSpec, '') <> ISNULL(i.TargetSpec, '')
        OR ISNULL(d.ParameterMap, '') <> ISNULL(i.ParameterMap, '');
END;
```

💡 Qué dice este trigger:

“Cada vez que alguien hace UPDATE a una fila de Operation, ANTES de aceptarlo, copio la versión vieja a la tabla `OperationHistory` con la fecha en que dejó de ser válida (`ValidTo = ahora`) y el usuario SQL que hizo el cambio (`SUSER_SNAME()`). Si nadie cambió `TargetSpec` ni `ParameterMap`, no escribo nada (UPDATE de campos accesorios no genera versión nueva).”

“Eso significa que en 6 meses, puedo reconstruir el prompt EXACTO que se usó el 14 de junio de 2026: `SELECT TargetSpec FROM OperationHistory WHERE OperationKey='hr.summarize_policy' AND ValidFrom <= '2026-06-14' AND '2026-06-14' < ValidTo`.”

Eso es responsable AI con propiedad de verdad. No es marketing.

🟢 **Objetivo AZ-500 aprobado en Mika:** Audit log inmutable, CorrelationId trazable, versionado automático del catálogo con triggers SQL — todo built-in, no opt-in.

13.4 — Failover como control de seguridad (BCDR para AI)

Objetivo AZ-500: “Implement business continuity and disaster recovery.”

En AZ-204 y AI-102 hablamos del failover desde el ángulo “no se cae”. AZ-500 lo examina desde el ángulo **“si el proveedor de tu AI ejerce control político sobre ti, ¿qué haces?”**.

Esto suena paranoico hasta que pasa. Cuando OpenAI restringió GPT-4 vision en cierta región. Cuando Anthropic cambió la política de uso a la mitad de un contrato. Cuando un proveedor decidió que cierto tipo de uso ya no estaba permitido. **Todas esas son “caídas” que NO son técnicas — son contractuales o políticas.**

El failover multi-vendor de Mika protege contra TODAS estas formas de “caída”:

Tipo de caída	Multi-region failover	Multi-vendor failover (Mika)
Caída de región	✓	✓
Rate limit del vendor	✗ (mismo vendor)	✓
Quota mensual agotada	✗	✓
Vendor sube precio inesperadamente	✗	✓
Vendor cambia ToS y prohíbe tu use case	✗	✓
Vendor deprecia el modelo	✗	✓
Vendor cierra operaciones en tu país	✗	✓

Eso es “BCDR para AI providers” — y NO es algo que el examen estándar AZ-500 examine explícitamente. Pero es **lo que la mentalidad AZ-500 te lleva a construir** si la aplicas en serio.

● **Objetivo AZ-500 aprobado en Mika** (*extendido a BCDR de proveedores AI*): Multi-vendor failover como control de continuidad — no solo multi-region.

13.5 — Cost as a security control: per-vendor caps

Objetivo AZ-500: *“Implement cost management and quota enforcement.”*

Este es uno donde la mayoría de apps fallan. AZ-500 enseña que **cost overruns son un riesgo de seguridad** — porque un atacante que invoque tu endpoint en loop puede agotar tu quota y dejarte sin AI por el resto del mes (denial of wallet).

En Mika el plan es **per-vendor caps** en la tabla [DataSource](#) :

```
-- Conceptual: la columna MonthlyTokenCap
ALTER TABLE DataSource
  ADD MonthlyTokenCap INT NULL,
  CurrentMonthTokens INT NULL DEFAULT 0;
```

Y el `executor.py` ANTES de invocar el LLM, verifica:

```
# Conceptual
if data_source.monthly_token_cap is not None:
    if data_source.current_month_tokens >= data_source.monthly_token_cap:
        # Cap reached: fail or fallback
        raise QuotaExceededError(...)
```

● **Honest declaration:** Esta lógica está **declarada en la arquitectura** y en el roadmap, pero la enforcement automática todavía no está wireada al executor en producción. El audit ya captura tokens — falta la capa de cap check.

● **Objetivo AZ-500 aprobado en Mika** (*declarado, no fully wireado*): Cost-as-security framework presente en el schema, listo para activar cuando llegue el primer cliente con SLA real.

13.6 — Compliance ready: “responsible AI” por construcción

Objetivo AZ-500: *“Apply responsible AI and compliance principles.”*

Esta es la sección más importante para vender a clientes enterprise en LATAM (financieros, salud, gobierno). El cliente NO te va a preguntar “¿qué modelo usas?”. Te va a preguntar:

- “¿Puedes mostrarme qué dijo la AI en este caso específico hace 6 meses?”
- “¿Puedes mostrarme quién aprobó este prompt?”
- “¿Puedes mostrarme cuántas veces se llamó esta operación este mes?”
- “Si auditoría externa nos audita, ¿puedes generar el reporte?”

Si la respuesta es “déjame revisar logs de varios servicios” — perdiste. Si la respuesta es “**sí, dame el CorrelationId o el rango de fechas y te traigo un PDF en 5 minutos**” — ganaste el contrato.

Mika está construida para que la respuesta sea siempre la segunda. **Eso es lo que AZ-500 te enseña a construir cuando dejas de ver seguridad como módulo y empiezas a verla como propiedad.**

● **Objetivo AZ-500 aprobado en Mika:** Compliance-ready by construction. Cada llamada tiene CorrelationId trazable; cada prompt tiene versión; cada cambio tiene historia; cada token cuenta.

14. La comparativa: Mika with vs without Azure

Mucha gente me pregunta: “¿Mika solo corre en Azure? ¿Por qué no AWS, GCP, Cloudflare?”

La respuesta honesta es: **Mika PUEDE correr en cualquier nube. Pero PIERDE cosas concretas si la desplazas fuera de Azure.** Aquí está la tabla:

Capa de Mika	En Azure	En AWS	En GCP	En self-host
FastAPI app	App Service / Container Apps	ECS / Lambda	Cloud Run	Docker en VM
SQL	Azure SQL (con Always Encrypted, TDE)	RDS SQL Server	Cloud SQL	SQL Server local
Secrets	Key Vault + Managed Identity	Secrets Manager + IAM	Secret Manager + IAM	HashiCorp Vault
Audit log	Built-in en SQL Server (mismo en todos)	Mismo	Mismo	Mismo
Multi-vendor LLM	Mismo (Mika es vendor-agnostic)	Mismo	Mismo	Mismo
Monitor	App Insights opcional	CloudWatch	Cloud Monitoring	Prometheus + Grafana
RBAC en código	Mismo (vive en <code>Principal</code>)	Mismo	Mismo	Mismo
Identidad usuario	Entra ID (canónico)	Cognito	Cloud Identity	Auth0 / Keycloak

💡 Qué dice esta tabla:

"La arquitectura de Mika (catálogo + executor + audit + multi-vendor) NO depende de Azure. Es portable. Lo que SÍ depende de Azure son las primitivas de infrastructure (Key Vault con MI, App Service, Azure SQL) — pero cada una tiene equivalente en AWS y GCP.

Lo único que cambia es el proveedor de identidad y secret store. Y como ya manejamos eso vía `DefaultAzureCredential` + `KeyVaultSecretResolver`, cambiar a AWS sería: una clase nueva `AwsSecretsManagerResolver`, swap del IAM role, y listo.

Mi decisión de Azure NO es técnica — es comercial. Los clientes LATAM enterprise que pago primero usan Azure y SQL Server. Por eso construyo donde están ellos."

15. Cierre Parte 3 + lo que viene en Parte 4

Lo que cubrió esta parte

1.  **AZ-204 — Cómo Mika CORRE:** FastAPI deployable en cualquier compute, SQL Server hardened, business-level monitoring built-in, exposición dual REST + MCP.
2.  **AZ-500 — Por qué Mika es SEGURA:** RBAC en Principal table, Managed Identity end-to-end, audit log immutable con CorrelationId, failover multi-vendor como BCDR, cost-as-security caps, compliance by construction.
3.  **Comparativa with/without Azure:** arquitectura portable; las primitivas de Azure son swappable; la decisión de stack es comercial, no técnica.

Lo que NO cubrí (audit-honest)

Cosas que las certs examinan pero Mika todavía no implementa fully:

- **Entra ID auth real** — `PrincipalId=1` sigue hardcoded. La estructura RBAC está; la integración Entra está en roadmap Q3.
- **Private Endpoints en Azure SQL y Key Vault** — la red sigue siendo public endpoint con firewall rules. AZ-500 canon dice Private Endpoint; aún no aplicado.
- **Defender for Cloud + Sentinel** — no habilitado. Para sandbox de desarrollo está OK, para producción es no negociable.
- **Always Encrypted en columnas PII** — `Candidato.rfc` , `Candidato.curp` deberían ser Always Encrypted. Hoy son `NVARCHAR` plano (sandbox).
- **Per-vendor cap enforcement automática** — declarada en arquitectura; falta el check en executor.

Declarar estos huecos es DELIBERADO. Es el mismo principio audit-honest de Parte 1 y 2: la diferencia entre Mika y un wrapper de ChatGPT incluye **declarar lo que no funciona** además de lo que sí.

Lo que viene en la Parte 4 (próximamente, 17 jun)

- **Tutorial step-by-step:** cómo replicar la arquitectura mínima de Mika tú mismo (sin tener que clonar el repo entero).
- **Glosario completo:** cada término técnico de las 3 partes, explicado para no-técnicos.
- **FAQ honest:** las 10 preguntas que más me han hecho sobre Mika (incluyendo las incómodas).
- **Próximos pasos:** roadmap real Q3-Q4 2026 y dónde estoy buscando feedback.

La frase de cierre

AI-102 me enseñó a hablar con Als. AZ-204 me enseñó a construir el backend que las hostea. AZ-500 me enseñó a no traicionar la confianza del cliente.

Mika es lo que pasa cuando las tres se aplican al mismo tiempo en un mismo sistema — sin tomar atajos.

Última actualización: 2026-06-15 06:30 CDMX · Parte 3 de 4 Código referenciado: `mika-core/api/main.py`, `mika-core/api/llm_runner.py`, `mika-core/providers/keyvault.py`, `mika-core/db/024_executionlog_llm_columns.sql`, `mika-core/db/037_operation_versioning.sql`
